PL/SQL (III)

Ș.L. Dr. Ing. Ciprian-Octavian Truică

ciprian.truica@upb.ro

As. Drd. Ing. Alexandru Petrescu

Alex.petrescu@upb.ro



Overview - Composite data types

Records

Collection Types: Associative Arrays, VARRAYS, Nested Tables

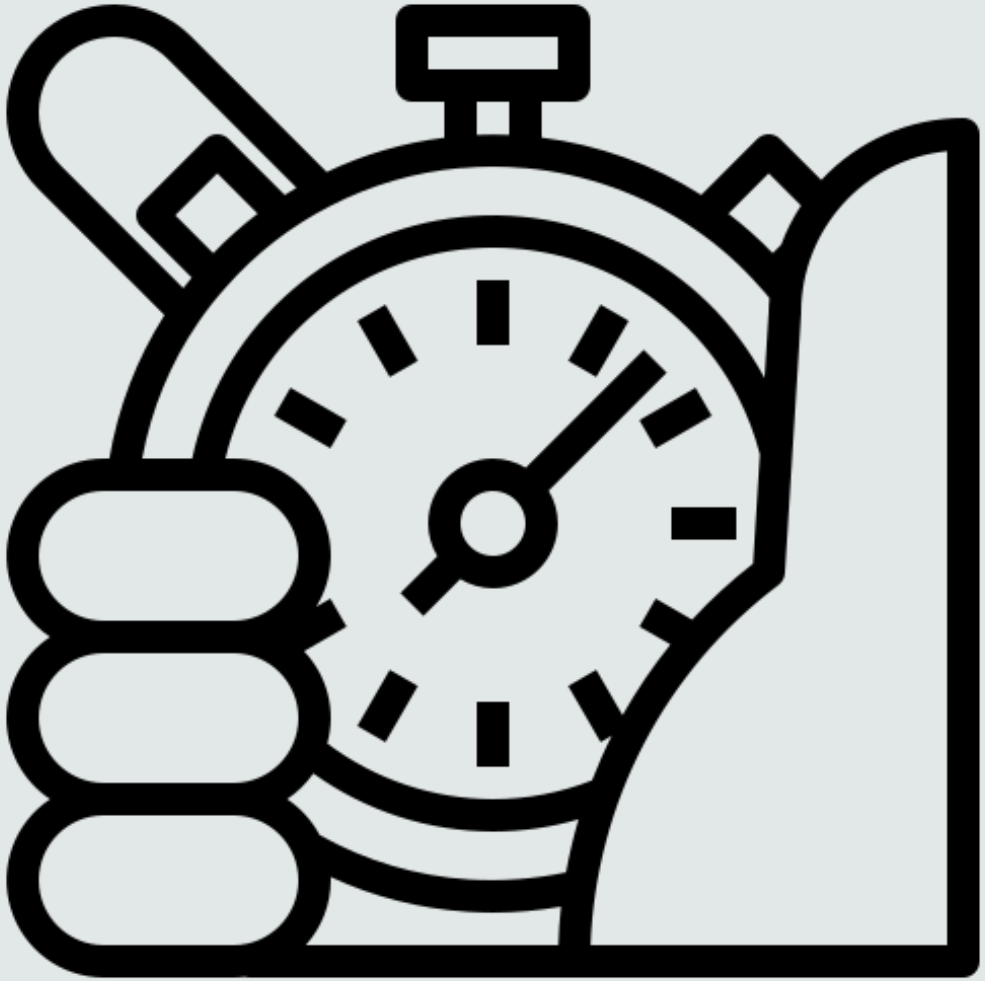
Collection Features

Composite data types

A **composite** data type stores values that have **internal components**. PL/SQL lets you define two kinds of composite data types:

- **Record**
- **Collection**

A **composite variable** can be passed to **subprograms** as **parameters** and its **internal components** can be accessed **individually**. Internal components can be either **Scalar** or **Composite**.



RECORD

Type

Records

The **internal components** of a **record** can have **different** data types and are called **fields**.

Each **field** of a record variable can be accessed by its name, e.g., variable_name.field_name.

Record variables can be:

- **Copies** of existing record variables
  - Created using the attribute **%ROWTYPE** or **%TYPE**
  - Inherits the only **structure**, **not** the **values**
- **User defined**
  - Define a **RECORD** type
  - Create a **variable** of that type

```
DECLARE
    TYPE rec1 IS RECORD(
        id countries.country_id%TYPE,
        name varchar2(20),
        reg regions%ROWTYPE
    );
    r1 rec1;
    r2 r1%type;
BEGIN
    r2.id := 1;
    r2.name = 'Hi, My Name is';
    dbms_output.put_line(r2.id);
END;
/
```

Records

A **RECORD** type defined in a PL/SQL **block** is a **local** type available **only** in the **block.** It is **stored** in the **database** only if the block is in a **subprogram** or **package**.

A **RECORD** type defined in a **package specification** is a **public** item and can be referenced from **outside** the **package** by qualifying it with the package name (package_name.type_name). It is **stored** in the database until you **drop** the **package** with the **DROP PACKAGE** statement.

A **RECORD** type cannot be created at **schema level** (a RECORD type cannot be an ADT attribute data type).

Records

To define a **RECORD** type, specify the record's **name** and define its **fields**. To define a **field**, specify the field's **name** and a its data **type**. The **initial** value of a **field** is **NULL**. The **NOT NULL** constraint can be specified for a field (a non-NULL **initial value** must also be **specified**).

A **non-NULL** initial value is **optional without** the **NOT NULL** constraint.

A **RECORD** type defined in a **package specification** is **incompatible** with an identically defined **local RECORD** type.

A **field** declared with **%ROWTYPE** or **%TYPE** still has an **initial NULL** value. (the variable **does not inherit** the initial **value** of the referenced item)


DO'S
DON'TS

Records

Records **cannot** be tested natively for **nullity**, **equality**, or **inequality**.

These BOOLEAN expressions are **illegal**:

- My_Record **IS NULL**
- My_Record_1 **=** My_Record_2
- My_Record_1 **>** My_Record_2

Records

Record variables are **allowed** only in these places:

- On the **right side** of the **SET** clause in an **UPDATE** statement (**Only 1** SET clause is allowed in this case). This can be combined with the "**ROW**" keyword on the **left side**.
- In the **VALUES** clause of an **INSERT** statement
- In the **INTO** subclause of a **RETURNING** clause

Record variables are **not allowed** in a **SELECT** list, a **WHERE** clause, a **GROUP BY** clause or in an **ORDER BY** clause. The "**ROW**" keyword **cannot** be used with a **subquery**.

Records

The following **restrictions** apply to **record inserts** and **updates**:

- If the **VALUES** clause of an **INSERT** statement contains a **RECORD** variable, **no other variable** or **value** is allowed in the **clause**
- If the **INTO** subclause of a **RETURNING** clause contains a **RECORD** variable, **no other variable** or **value** is allowed in the **subclause**

These are **not supported**:

- **Nested RECORD** types
- **Functions** that **return** a **RECORD** type
- Record inserts and updates using the **EXECUTE IMMEDIATE** statement (Dynamic SQL)

PL/SQL engine cares about type compatibility so as long as our instructions leverage the proper types, all is good.

```
set serveroutput on
DECLARE
  TYPE rec1 IS RECORD(
    id countries.country_id%TYPE,
    name varchar2(20),
    reg regions%ROWTYPE
  );
  r1 rec1;
  r2 countries%ROWTYPE;
  r3 countries%ROWTYPE;
BEGIN
  SELECT * INTO r1.reg FROM regions
  WHERE lower(region_name) LIKE '%rop_';

  r1.id := 'UA';
  r1.name := 'Ukraina';

  INSERT INTO countries
  VALUES(r1.id, r1.name, r1.reg.region_id);


  SELECT * INTO r2 FROM countries
  WHERE country_id = r1.id;

  r2.country_id := 'ME';
  r2.country_name := 'Moldova';
  r2.region_id := r1.reg.region_id;

  INSERT INTO countries VALUES r2
  RETURNING country_id, country_name, region_id into r3;

  dbms_output.put_line('====INSERT====');
  dbms_output.put_line(r2.country_id || ' ' || r2.country_name || ' ' || r2.region_id);
  dbms_output.put_line(r3.country_id || ' ' || r3.country_name || ' ' || r3.region_id);


  r2.country_id := 'MD';
  r2.country_name := 'Moldova';
  r2.region_id := r1.reg.region_id;

  UPDATE countries SET ROW = r2 WHERE country_id = 'MD'
  RETURNING country_id, country_name, region_id into r3;

  dbms_output.put_line('====UPDATE====');
  dbms_output.put_line(r2.country_id || ' ' || r2.country_name || ' ' || r2.region_id);
  dbms_output.put_line(r3.country_id || ' ' || r3.country_name || ' ' || r3.region_id);

  DELETE FROM countries WHERE country_id = 'MD'
  RETURNING country_id, country_name, region_id into r3;

  dbms_output.put_line('====DELETE====');
  dbms_output.put_line(r2.country_id || ' ' || r2.country_name || ' ' || r2.region_id);
  dbms_output.put_line(r3.country_id || ' x ' || r3.country_name || ' ' || r3.region_id);

END;
/
```

Collection Types

Collection types

The **internal components** of a **collection** have the **same** data **type** and are called **elements**. Elements of a **collection** variable can be accessed by its **unique index**, e.g., variable_name(**index**)

Collections variables can be:

- **Copies** of **existing** composite variables (created using the attribute %TYPE - Inherits the only structure, not the values)
- **User defined** – define a collection type and then create a variable of that type

Collection types

Collections have **one dimension**. A **multidimensional** collection can be modeled by using a **collection** whose **elements** are **collections**. Developers can create:

- A **collection** of **records**: a **collection** that has **elements records** and accessing an **item** "variable_name(**index**).field_name"
- A **record** that contains **collections**: a **record** that has **items collections** and accessing a **field** "variable_name.field_name(**index**)"
- A **collection** of **collections**: a **collection** that has items **a collection** and accessing an item "variable_name(**index_1**)(**index_2**)⋯(**index_n**)"

Collection types

Collection Type	Number of Elements	Index Type	Dense or Sparse	Uninitialized Status	Defined in	ADT Type Attr?
Associative array (or index-by table)	Unspecified	**String** **PLS_INTEGER** (BINARY_INTEGER)	Either	Empty	In PL/SQL block or package	No
VARRAY (variable-size array)	Specified	Integer	Always dense	Null	In PL/SQL block or package or at schema level	Only defined at schema level
Nested table	Unspecified	Integer	Starts dense, can become sparse	Null	In PL/SQL block or package or at schema level	Only defined at schema level

Number of **Elements**, **Index**, **Dense** or **Sparse**

Number of **Elements**?

- **Specified**: the **maximum** number of elements in the **collection**
- **Unspecified**: the **maximum** number of elements in the collection is the **upper limit of the index type**

Dense or **Sparse**?

- A **dense** collection has **no gaps** between **elements**, every element between the **first** and **last** element is **defined** (meaning it has a value, the value can be **NULL** unless the element has a **NOT NULL** constraint).
- A **sparse** collection **has gaps** between elements.

Uninitialized Status

An **empty** collection **exists** but has **no elements**. The **EXTEND** method must be invoked to **add elements** to an **empty** collection.

A **null** collection (also called an **atomically null** collection) **does not exist**. The collection must be initialized to change it from a null collection to an existing collection. The **EXTEND** method cannot be used to initialize a null collection. Initialization:

- Making it empty
- Assigning a non-NULL value to it

Defined in

Same as for **RECORD**: if it's in a **block**, it is **local** / if it's in a **program specification**, it is **public**. Can be **stored** as part of a **subprogram** else it is gone with the session.

A **collection** type **defined** at **schema level** is a standalone type. It is **stored** in the database until you **drop** it with the **DROP TYPE** Statement.

- Can be **created** with the **CREATE** [**OR REPLACE**] **TYPE**
- Can be **modified** using the **ALTER TYPE**

A collection type defined in a package specification is incompatible with an identically defined local or standalone collection type.

ADT Attribute Data Type

To be an **ADT**(Abstract Data Type) **Attribute** Data Type, a collection type must be a **standalone** collection type. Translating Non-PL/SQL Composite Types to PL/SQL Composite Types:

Non-PL/SQL Composite Type	Equivalent PL/SQL Composite Type
Hash table	Associative array
Unordered table	Associative array
Set	Nested table
Bag	Nested table
Array	VARRAY



Associative Arrays

Associative Arrays

An **associative array** (formerly called **PL/SQL table** or **index-by table**) is a **set** of **key-value** pairs where each **key** is a **unique index**. The **index** is used to locate the associated **value** with the syntax "variable_name(**index**)".

An **associative array** (like a database table) is **empty** (but not null) until it is **populated**. It can **hold** an **unspecified** number of elements (accessible without knowing their positions). Unlike a database table, it **does not** need **disk space** and **cannot** be manipulated with **DML** statements .

Associative Arrays - Indexes

The data type of **index** can be:

- A **string** type (**VARCHAR2**, **VARCHAR**, **STRING**, or **LONG**)
- **PLS_INTEGER**/**BINARY_INTEGER**

Indexes are **stored** in **sort order**, but are **not created** in **order**. The **sort order** is determined by the initialization parameter **NLS_SORT** and **NLS_COMP** for string types. National Language Support (**NLS**) parameters such as **NLS_SORT**, **NLS_COMP**, and **NLS_DATE_FORMAT** affect associative arrays indexed by **string**.

Associative Arrays

An **associative array** is **appropriate for**:

- A **relatively small** lookup table constructed in **memory** each time the subprogram is invoked or the package that declares it is initialized.
- To pass **collections** to and from the **database server**: Declare formal **subprogram parameters** of **associative array** types > Bind the **host arrays** to the corresponding actual **parameters** (use Oracle Call Interface (**OCI**) or an Oracle Precompiler) > PL/SQL **automatically converts** between host arrays and associative arrays indexed by **PLS_INTEGER** (You **cannot bind** an associative array **indexed** by **VARCHAR**)

Associative Arrays

The **associative array** type **cannot** be **declared** at **schema level** as it is intended for **temporary data storage**. To make an **associative array persistent** declare it in a **package specification** or populate it in the **package body**.

To pass an **associative array** variable as a **parameter** to a standalone **subprogram** the type of that variable must be declared in a **package specification**. Doing so makes the type **available** to the **invoked subprogram** (which declares a formal parameter of that type) and the **invoking subprogram** or anonymous block (which declares and passes the variable of that type).

The **most efficient** way to pass **collections** to and from the database server is to use **associative arrays** with **FORALL** statement and **BULK COLLECT** clause.

```
set serveroutput on
DECLARE
  TYPE rec IS RECORD(
    country_id countries.country_id%TYPE,
    country_name countries.country_name%TYPE,
    region_id countries.region_id%TYPE
  );
  TYPE aa IS TABLE OF rec INDEX BY PLS_INTEGER;

  v aa; -- create a null associative array
  r rec;

  idx PLS_INTEGER;

  TYPE idxs_aa IS TABLE OF PLS_INTEGER INDEX BY PLS_INTEGER;
  idxs idxs_aa;

  i PLS_INTEGER := 1;
BEGIN
  r.country_id := 'UA';
  r.country_name := 'Ukraina';
  r.region_id := 1;

  v := aa(1 => r, 2 => rec('MD', 'Moldova', 1));

  v(5).country_id := 'BG';
  v(5).country_name := 'Bulgaria';
  v(5).region_id := 1;

  idx := v.FIRST;
  WHILE idx IS NOT NULL
  LOOP
    dbms_output.put_line(idx||' '||v(idx).country_id||' '
                         ||v(idx).country_name||' '||v(idx).region_id);
    idxs(i) := idx;
    idx := v.next(idx);
    i := i + 1;
  END LOOP;

  FORALL j IN VALUES OF idxs INSERT INTO COUNTRIES VALUES v(j);

END;
/
```

```
DECLARE
  TYPE rec IS RECORD(
    country_id countries.country_id%TYPE,
    country_name countries.country_name%TYPE,
    region_id countries.region_id%TYPE
  );

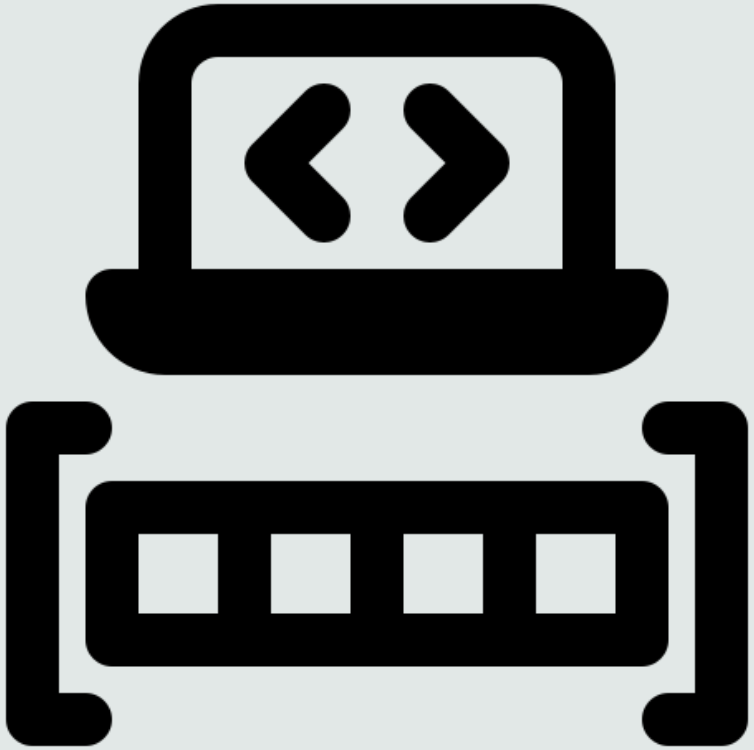
  TYPE aa IS TABLE OF rec INDEX BY PLS_INTEGER;
  v aa := aa(); -- create an empty associative array;
  r rec;
  idx PLS_INTEGER;
BEGIN
  r.country_id := 'RO';
  r.country_name := 'Romania';
  r.region_id := 1;

  v(1) := r;
  v(2) := rec('MD', 'Moldova', 1);

  v(3).country_id := 'BG';
  v(3).country_name := 'Bulgaria';
  v(3).region_id := 1;

  SELECT * BULK COLLECT INTO v from countries order by country_id desc;

  idx := v.FIRST;
  WHILE idx IS NOT NULL
  LOOP
    dbms_output.put_line(idx||' '||v(idx).country_id||' '||v(idx).country_name||' '||v(idx).region_id);
    idx := v.next(idx);
  END LOOP;
END;
/
```



VARRARAYS

VARRARYS

A **varray** (**variable-size** array) is an array whose **number** of **elements** can vary from zero (empty) to the **declared** maximum size. An **uninitialized** varray variable is a **null collection**. A **varray** variable must be **initialized** by either making it **empty** or assigning a **non-NULL** value to it.

A **varray** is **appropriate** when the maximum number of needed elements is **known** and the elements are **accessed sequentially**. A **varray** might be **impractical** for storing **many** elements because all elements must be stored or retrieved at the same time.

VARRARYS - Index

Use the syntax "variable_name(index)" to **access** an **element** of a **varray** variable, where the **lower bound** of **index** is **1** and the **upper bound** is the **current number** of **elements**.

The **upper bound changes** as you add or delete **elements but** cannot exceed the maximum size.

```
DECLARE
  TYPE vec1 IS VARRAY(150) OF countries.country_id%type;
  v1 vec1 := vec1('RO', 'MD');
  TYPE vec2 IS VARRAY(150) OF countries.country_name%type;
  v2 vec2 := vec2('Romania', 'Moldova');
BEGIN
  FORALL i IN v1.FIRST .. v1.LAST
    INSERT INTO countries(country_id) VALUES(v1(i));

  FORALL i IN v2.FIRST .. v2.LAST
    UPDATE countries SET country_name=v2(i), region_id=1 WHERE country_id = v1(i);

  SELECT country_id, country_name
  BULK COLLECT INTO v1, v2
  FROM countries
  WHERE country_id NOT IN (SELECT DISTINCT country_id FROM locations WHERE country_id IS NOT
NULL);
  FOR i IN v1.FIRST .. v1.LAST
  LOOP
    dbms_output.put_line(v1(i)||' '||v2(i));
  END LOOP;

  FORALL i IN v1.FIRST .. v1.LAST
    DELETE countries WHERE country_id = v1(i);

END;
/
```



Nested Tables

Nested Tables

A **nested table** is a collection type that stores an **unspecified number** of rows in **no particular order**. When retrieving a **nested table** value from the **database** PL/SQL gives the rows **consecutive indexes** starting at **1**.

The syntax is "variable_name(**index**)".

A **nested table**'s **indexes** and **row order** might **not remain stable** when **storing** or **retrieving** the **nested table** to and from the **database**.

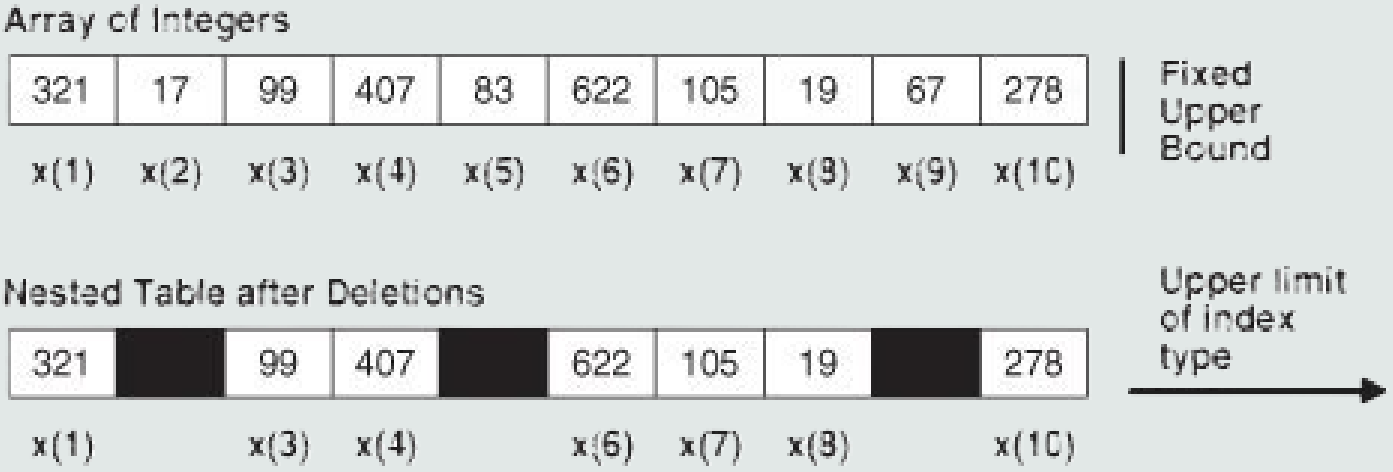
The amount of **memory** that a nested table variable occupies can **increase** or **decrease dynamically** as elements are added or deleted.

An **uninitialized** nested table variable is a **null collection**. A **nested table** is initialized by either making it **empty** or assigning a **non-NULL** value to it.

Nested Tables

Conceptually a **nested table** is like a one-dimensional **array** with an **arbitrary** number of elements, however, there are **differences**:

- The **size** of a nested table can **increase dynamically**: an **array** has a **declared** (**maximum**) number of elements, a nested table **does not**
- A **nested table** is **dense initially** but can become **sparse** as elements are **deleted** (at arbitrary indexes), whereas the **array** remains **dense**



Nested Tables

A **nested table** is **appropriate** when:

- The (maximum)number of elements is **not set**
- **Index** values are **not consecutive**
- To **delete** or **update some elements**, but **not all** elements **simultaneously**

```
DECLARE
  TYPE tbl1 IS TABLE OF countries.country_id%type;
  t1 tbl1 := tbl1('RO', 'MD');
  TYPE tbl2 IS TABLE OF countries.country_name%type;
  t2 tbl2 := tbl2('Romania', 'Moldova');
BEGIN
  FORALL i IN INDICES OF t1 INSERT INTO countries(country_id) VALUES(t1(i));

  FORALL i IN INDICES OF t2 UPDATE countries SET country_name=t2(i), region_id=1 WHERE country_id =
t1(i);

  SELECT country_id, country_name
  BULK COLLECT INTO t1, t2 FROM countries
  WHERE country_id NOT IN (SELECT DISTINCT country_id FROM locations WHERE country_id IS NOT NULL);

  FOR i IN t1.FIRST .. t1.LAST
  LOOP
    dbms_output.put_line(t1(i)||' '||t2(i));
  END LOOP;

  FORALL i IN INDICES OF t1 DELETE countries WHERE country_id = t1(i);

END;
/
```

Collection Features



Collection Constructors

A **collection constructor** (constructor) is a system-defined **function** that has the **same name** as a collection type and **returns** a collection of that type.

```
-- declare the type

TYPE vec IS VARRAY(10) OF INTEGER;

-- initialize an empty variable

var vec := vec();
```



IT'S DANGEROUS TO GO ALONE! TAKE THIS.

Assigning Values to Collection Variables

- **Constructor**: Invoke a **constructor** to **create** a collection and **assign values** to the collection **elements**
- **Assignment**: To assign the value of **another** existing **collection** variable
- **Subprogram**: **Pass** the collection variable to a **subprogram** / Use the **OUT** or **IN OUT** parameter / **Assign** the value **inside** the **subprogram**

A collection can be assigned to a collection variable if they have the same data type. Having the same element type is not enough for an assignment.

Assigning Values to Collection Variables

To make a **collection** variable **null** assign either a **NULL value** or a **NULL collection** of the same data type.

The **SQL MULTISET** operators (MULTISET **EXCEPT**, MULTISET **INTERSECT**, MULTISET **UNION**) **combine** two **nested tables** into a **single** nested table. The elements of the two **nested table**s must have **comparable** data types.

The result of the **SQL SET** function invocation takes a **nested table** argument and returns a nested table of the **same data type** whose elements are **distinct**. The function **eliminates duplicate** elements.

```
DECLARE
    TYPE t IS TABLE OF INTEGER;
    s1 t := t(1,1,2,2,3,4);
    s2 t := t(2,4,4,6,6, 8, 8);
    s3 t;
    s4 t := t(3, 4);
    PROCEDURE print(x t) IS
      s string(100) := '';
    BEGIN
        FOR i IN x.first .. x.last
        LOOP
            s := s||x(i)||' ';
        END LOOP;
        dbms_output.put_line(s);
    END print;
BEGIN
    IF s3 IS NULL THEN
        dbms_output.put_line('EMPTY AND NULL');
    ELSIF s3 IS NOT NULL AND s3 IS EMPTY THEN
        dbms_output.put_line('EMPTY AND NOT NULL');
    ELSIF s3 IS NOT NULL AND s3 IS NOT EMPTY THEN
        dbms_output.put_line('NOT EMPTY AND NOT NULL');
    END IF;

    dbms_output.put_line('UNION');
    s3 := s2 MULTISET UNION S1;
    print(s3);

    dbms_output.put_line('SET');
    print(SET(s3));

    dbms_output.put_line('INTERSECT');
    s3 := s1 MULTISET INTERSECT S2;
    print(s3);

    dbms_output.put_line('EXCEPT');
    s3 := s1 MULTISET EXCEPT S2;
    print(s3);

    dbms_output.put_line('EXCEPT DISTINCT');
    s3 := s1 MULTISET EXCEPT DISTINCT S2;
    print(s3);

    dbms_output.put_line('EXCEPT');
    s3 := s2 MULTISET EXCEPT S1;
    print(s3);

    dbms_output.put_line('EXCEPT DISTINCT');
    s3 := s2 MULTISET EXCEPT DISTINCT S1;
    print(s3);

    IF s4 SUBMULTISET OF s1 THEN
        dbms_output.put_line('TRUE');
    ELSE
        dbms_output.put_line('FALSE');
    END IF;

    dbms_output.put_line(CARDINALITY(s1));
    dbms_output.put_line(CARDINALITY(SET(s1)));

    IF &VAL MEMBER OF S1 THEN
        dbms_output.put_line('TRUE');
    END IF;
END;
/
```

```
DECLARE
    TYPE t IS TABLE OF INTEGER;
    s1 t := t(1,1,2,2,3,4);
    s2 t := t(2,4,4,6,6, 8, 8);
    s3 t;
    s4 t := t(3, 4);
    card number(4);
BEGIN
    IF s3 IS NULL THEN
        dbms_output.put_line('EMPTY AND NULL');
    ELSIF s3 IS NOT NULL AND s3 IS EMPTY THEN
        dbms_output.put_line('EMPTY AND NOT NULL');
    ELSIF s3 IS NOT NULL AND s3 IS NOT EMPTY THEN
        dbms_output.put_line('NOT EMPTY AND NOT NULL');
    END IF;

    s3 := s2 MULTISET UNION S1; -- 2 4 4 6 6 8 8 1 1 2 2 3
4   s3 := SET(s3); -- 2 4 6 8 1 3
    s3 := s1 MULTISET INTERSECT S2; -- 2 4
    s3 := s1 MULTISET EXCEPT S2; -- 1 1 2 3
    s3 := s1 MULTISET EXCEPT DISTINCT S2; -- 1 3
    s3 := s2 MULTISET EXCEPT S1; -- 4 6 6 8 8
    s3 := s2 MULTISET EXCEPT DISTINCT S1; -- 6 8

     IF s4 SUBMULTISET OF s1 THEN
        dbms_output.put_line('TRUE');
    ELSE
        dbms_output.put_line('FALSE');
    END IF;

    card := CARDINALITY(s1); -- 6
    card := CARDINALITY(SET(s1)); -- 4

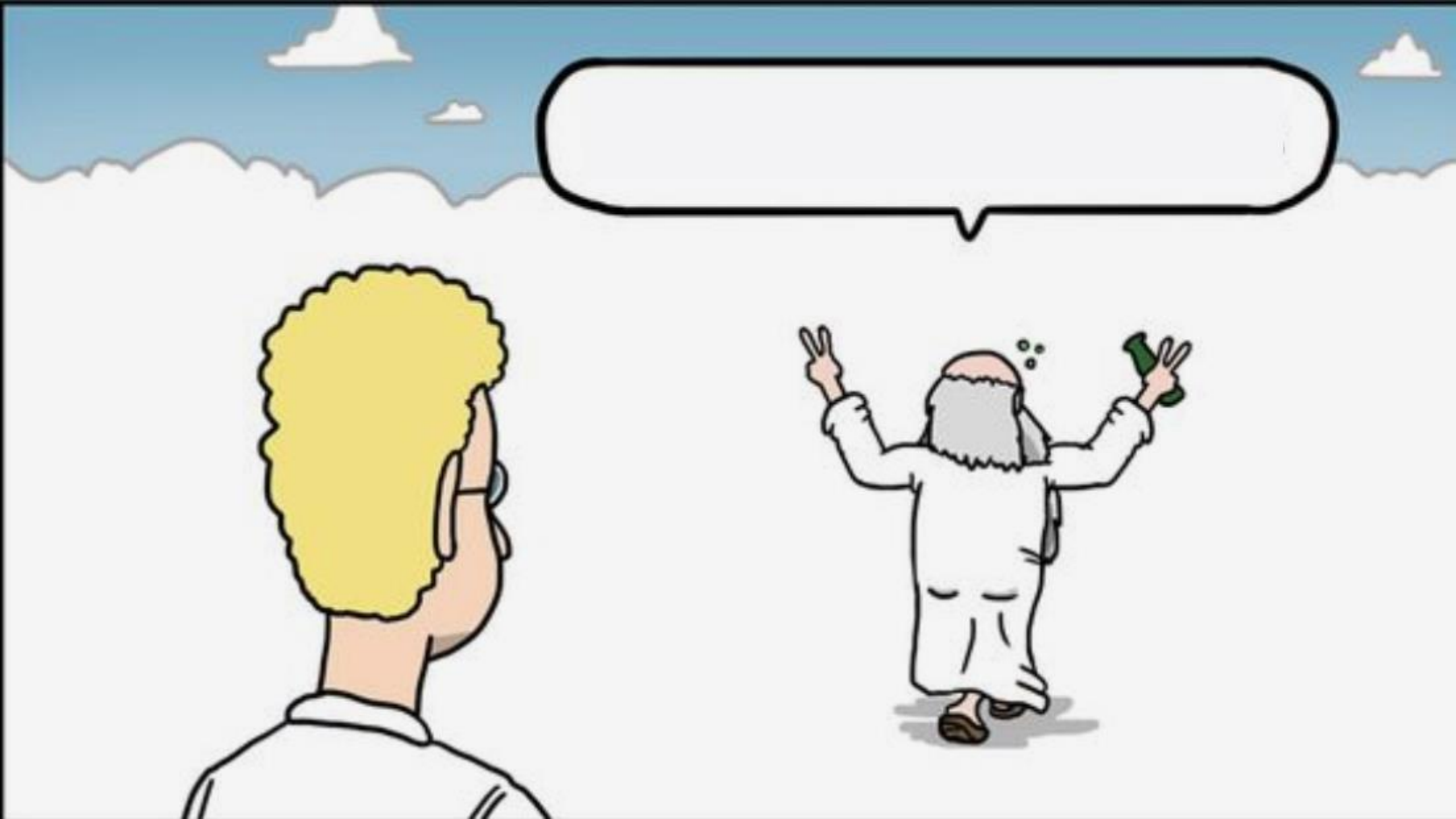
    IF &VAL MEMBER OF S1 THEN
        dbms_output.put_line('TRUE');
    END IF;
END;
/
```

Collection Methods

A **collection method** is a PL/SQL **subprogram**, either a **function** that **returns information** about a collection or a **procedure** that **operates** on a collection.

Collection methods make collections easier to use and applications easier to maintain. The syntax for a method invocation is "collection_name.method".

Method	Type	Description
DELETE	Procedure	Deletes elements from collection.
TRIM	Procedure	Deletes elements from end of varray or nested table.
EXTEND	Procedure	Adds elements to end of varray or nested table.
EXISTS	Function	Returns TRUE if and only if specified element of varray or nested table exists.
FIRST	Function	Returns first index in collection.
LAST	Function	Returns last index in collection.
COUNT	Function	Returns number of elements in collection.
LIMIT	Function	Returns maximum number of elements that collection can have.
PRIOR	Function	Returns index that precedes specified index.
NEXT	Function	Returns index that succeeds specified index.



Collection Methods - **DELETE**

DELETE is a **procedure** that removes elements from a **collection**. "**DELETE**" **removes all** elements from a collection of **any type**. This operation **immediately** frees the memory allocated by the deleted elements.

For an **associative array** or **nested table** (but **not** a **varray**):

- **DELETE**(n) **removes the** element whose **index** is **n** if that element exists, otherwise it does nothing
- **DELETE**(m,n) **removes all** elements whose **indexes** are in the **range** m..n if both m and n exist and m <= n, otherwise it does nothing

For the above forms of **DELETE** PL/SQL keeps **placeholders** for the deleted elements. The deleted elements are included in the internal size of the collection, and they can be "restored" by assigning a valid value to it. (the cell, not the value of the element)

```
DECLARE
  TYPE nt is TABLE OF NUMBER(4);
  t nt := nt(11, 22, 33, 44, 55, 66);
  i  NUMBER;
BEGIN
  t.DELETE(2);      -- Delete second element
  t(2) := 2222;     -- Restore second element
  t.DELETE(2, 4);   -- Delete range of elements
  i := t.FIRST;
  WHILE i IS NOT NULL LOOP
    dbms_output.put_line(t(i));
    i := t.NEXT(i);
  END LOOP;
  t(3) := 3333;     -- Restore third element
  t.DELETE;         -- Delete all elements
END;
/
```

```
DECLARE
  TYPE nt IS TABLE OF NUMBER(4) INDEX BY VARCHAR2(10);
  t nt := nt('a'=>1, 'b'=>2, 'c'=>30, 'd'=>4, 'e'=>5, 'f'=>6);
  i VARCHAR2(10);
BEGIN
  t('c') := 300;  -- Replace element
  t('g') := 7; -- Add a new element
  t.DELETE('c');
  t.DELETE('g');
  t('c') := 3; -- Replace deleted element with new value
  t('g') := 7; -- Replace deleted element with same value
  t('h') := 7; -- Add a new element
  t('i') := 7; -- Add a new element
  t('j') := 7; -- Add a new element
  t.DELETE('c','e');  -- Delete range of elements
  t.DELETE('j','f');  -- Does nothing

  i := t.FIRST;
  WHILE i IS NOT NULL LOOP
    dbms_output.put_line(i||' '||t(i));
    i := t.NEXT(i);
  END LOOP;

  t.DELETE; -- Delete all elements
END;
/
```

BEFORE





AFTER

Collection Methods - **TRIM**

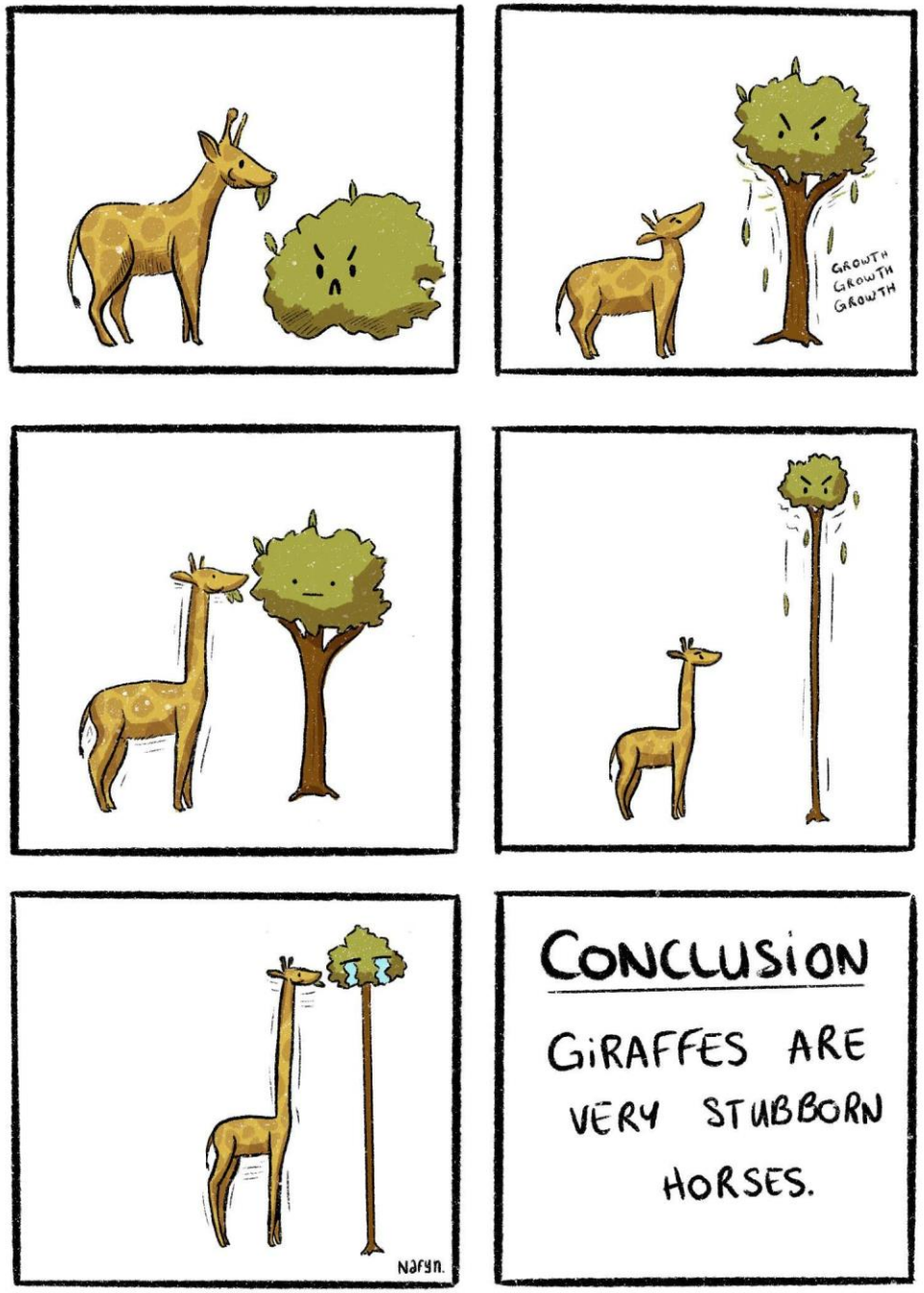
TRIM is a **procedure** that **deletes** elements from the **end** of a **varray** or **nested table**. This method has these forms:

- **TRIM** removes **one element** from the **end** of the collection if the collection has at least one element, otherwise it **raises** the predefined exception **SUBSCRIPT_BEYOND_COUNT**
- **TRIM**(n) removes **n elements** from the **end** of the collection if there are at least n elements at the end, otherwise it **raises** the predefined exception **SUBSCRIPT_BEYOND_COUNT**

TRIM operates on the **internal size** of a collection. It considers the element to exist: if **DELETE** removed an element but **kept** a **placeholder** for it, **TRIM** does a **clean wipe**. PL/SQL **does not** keep **placeholders** for **trimmed** elements. (Trimmed elements are not included in the internal size of the collection; a trimmed element cannot be restored by assigning a valid value to it)

```
DECLARE
  TYPE nt is TABLE OF NUMBER(4);
  t nt := nt(1, 2, 3, 4, 5, 6);
  i  NUMBER;
BEGIN
  t.TRIM; -- Trim last element
  t.DELETE(4); -- Delete fourth element
  t.TRIM(2); -- Trim last two element
  i := t.FIRST;
  WHILE i IS NOT NULL LOOP
    dbms_output.put_line(t(i));
    i := t.NEXT(i);
  END LOOP;
END;
/
```

GIRAFFE: THE GENESIS



Collection Methods - **EXTEND**

EXTEND is a **procedure** that **adds elements** to the **end** of a **varray** or **nested table**. The collection **can be empty**, but **not null** (Use a constructor to make a collection empty or add elements to a null collection). The **EXTEND** method has these forms:

- **EXTEND**: appends **one null** element to the end of the collection
- **EXTEND**(n): appends **n null** elements to the end of the collection
- **EXTEND**(n,i): appends **n copies** of the **i-th elemen**t to the collection. Is the **only** form that you can use for a collection whose elements have the **NOT NULL** constraint.

EXTEND operates on the **internal size** of a collection, it considers the element **to exist**, like **TRIM**, unlike **DELETE**.

```
DECLARE
  TYPE nt is TABLE OF NUMBER(4);
  t nt := nt(1, 2, 3, 4, 5, 6);
  i  NUMBER;
BEGIN
  t.EXTEND(2, 3); -- Append two copies of first element
  t.DELETE(8); -- Delete eighth element
  t.EXTEND; -- Append one null element
  i := t.FIRST;
  WHILE i IS NOT NULL LOOP
    dbms_output.put_line(i||' '||t(i));
    i := t.NEXT(i);
  END LOOP;
END;
/
```


It's Free Real Estate

Collection Methods - **EXISTS**

EXISTS is a **function** that tells you whether the specified element of a **varray** or **nested table** exists.

EXISTS(n): returns **TRUE** if the **n-th element** of the collection exists and **FALSE** otherwise.

If **n** is **out of range EXISTS** returns **FALSE** instead of raising the predefined exception **SUBSCRIPT_OUTSIDE_LIMIT**.

EXISTS(n) returns **FALSE** for a **deleted element** (even if **DELETE** kept a **placeholder** for it).

```
DECLARE
  TYPE rec IS RECORD(
      country_name countries.country_name%TYPE,
      region_name regions.region_name%TYPE
    );
  TYPE tbl IS TABLE OF rec;
  t tbl;
BEGIN

  SELECT country_name, region_name
  BULK COLLECT INTO t
  FROM countries natural join regions;

  dbms_output.put_line(t.FIRST); -- 1
  dbms_output.put_line(t.LAST); -- 25
  dbms_output.put_line(t.COUNT); -- 25

  t.DELETE(5, 15); -- Delete range

  FOR i IN t.FIRST .. t.LAST
  LOOP
    IF t.EXISTS(i) THEN -- test if exists
      dbms_output.put_line(t(i).country_name||' '||t(i).region_name);
    END IF;
  END LOOP;

  dbms_output.put_line(t.FIRST); -- 1
  dbms_output.put_line(t.LAST); -- 25
  dbms_output.put_line(t.COUNT); -- 14

END;
/
```


i am at my limit.

Collection Methods – **COUNT** and **LIMIT**

COUNT is a **function** that returns the **number of elements** in the **collection**. **COUNT ignores** deleted elements, even if **DELETE** kept **placeholders** for them.

For a **nested table COUNT** is smaller than **LAST** (the **index** of the **last** element) if elements from the **middle** of the nested table are **deleted**.

https://pt.memedroid.com/memes/detail/3965907/i-am-at-my-limit-too

LIMIT is a function that returns the **maximum number** of elements that the collection can have.

LIMIT returns **NULL** if the collection has no maximum number of elements. (Only a varray has a maximum size)



Collection Methods – **FIRST** and **LAST**

FIRST and **LAST** are functions. If the **collection** has at least one element **FIRST** return the **index** of the first element and **LAST** returns the **index** of the last element. They **ignore deleted** elements even if **DELETE** kept **placeholders**. **FIRST** and **LAST** return the same **index** if the collection has only one element. **FIRST** and **LAST** return **NULL** if the collection is **empty**.

- For an **associative array**, indexed by **PLS_INTEGER**: The **FIRST** element is the **smallest index** and the **LAST** element is the **largest index**
- For an **associative array**, indexed by **string**: The **FIRST** element is the **lowest key value** and the **LAST** element is the **highest key value**
- For a **varray** that is **not empty**: **FIRST** always returns **1** and **LAST** always equals **COUNT**
- For a **nested table**: **LAST** equals **COUNT** if elements from the **middle** of the nested table are **NOT deleted** and **LAST** is larger than **COUNT** if elements from its middle are **deleted**.

```
DECLARE
  TYPE rec IS RECORD(
      country_name countries.country_name%TYPE,
      region_name regions.region_name%TYPE
    );
  TYPE tbl IS TABLE OF rec;
  t tbl;
BEGIN

  SELECT country_name, region_name
  BULK COLLECT INTO t
  FROM countries natural join regions;

  dbms_output.put_line(t.FIRST); -- 1
  dbms_output.put_line(t.LAST); -- 25
  dbms_output.put_line(t.COUNT); -- 25

  t.DELETE(5, 15); -- Delete range

  FOR i IN t.FIRST .. t.LAST
  LOOP
    IF t.EXISTS(i) THEN -- test if exists
      dbms_output.put_line(t(i).country_name||' '||t(i).region_name);
    END IF;
  END LOOP;

  dbms_output.put_line(t.FIRST); -- 1
  dbms_output.put_line(t.LAST); -- 25
  dbms_output.put_line(t.COUNT); -- 14

END;
/
```

```
DECLARE
  TYPE tbl IS TABLE OF countries.country_name%TYPE;
  TYPE vec IS VARRAY(100) OF regions.region_name%TYPE;
  TYPE rec IS RECORD(
      country_names tbl,
      region_names vec
    );
  r rec;
BEGIN

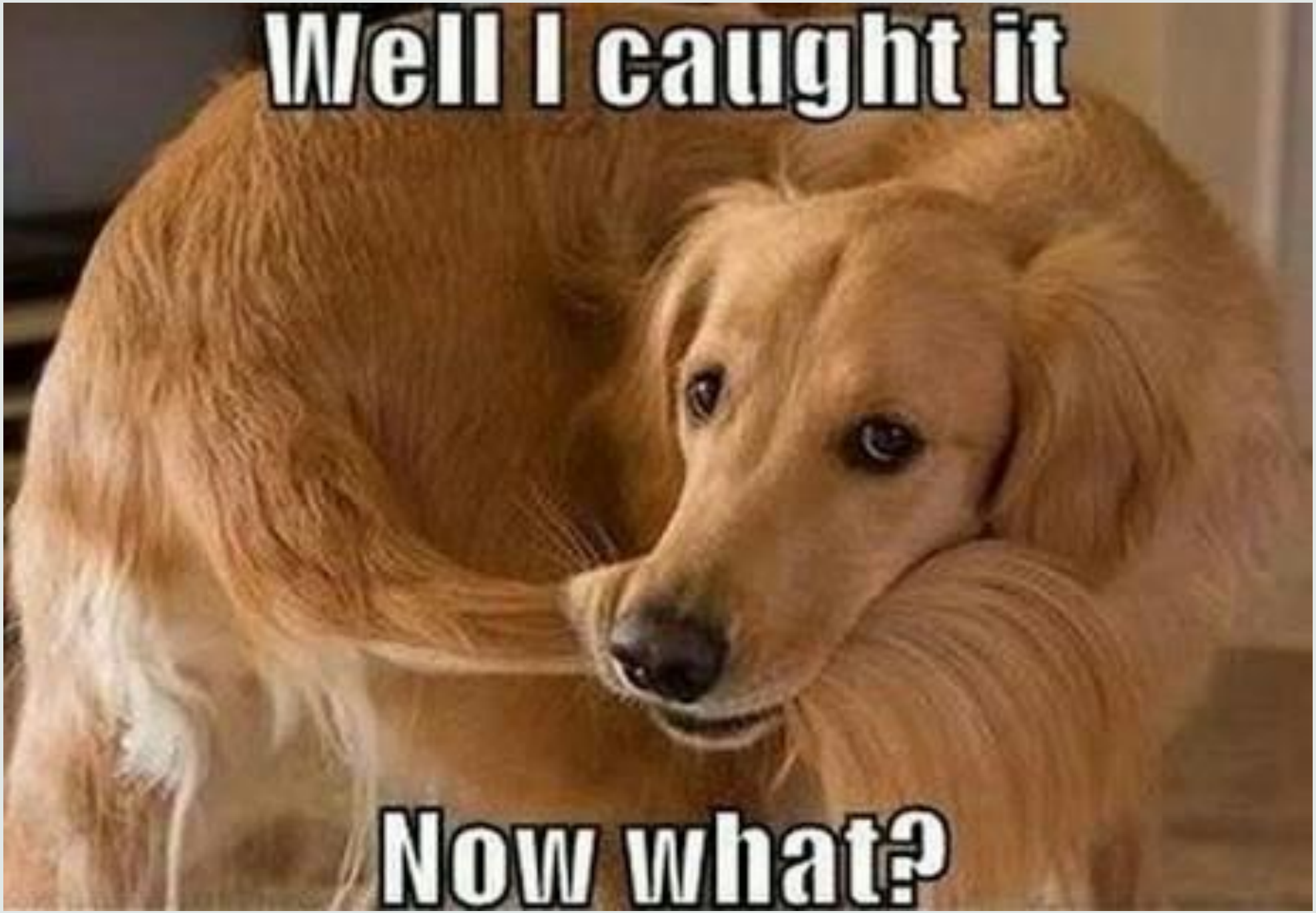
  SELECT country_name
  BULK COLLECT INTO r.country_names
  FROM countries;

  SELECT region_name
  BULK COLLECT INTO r.region_names
  FROM regions;

  dbms_output.put_line('country_names.FIRST '||r.country_names.FIRST); -- 1
  dbms_output.put_line('country_names.LAST '||r.country_names.LAST); -- 25
  dbms_output.put_line('country_names.COUNT '||r.country_names.COUNT); -- 25
  dbms_output.put_line('country_names.LIMIT '||r.country_names.LIMIT); -- NULL

  dbms_output.put_line('region_names.FIRST '||r.region_names.FIRST); -- 1
  dbms_output.put_line('region_names.LAST '||r.region_names.LAST); -- 4
  dbms_output.put_line('region_names.COUNT '||r.region_names.COUNT); -- 4
  dbms_output.put_line('region_names.LIMIT '||r.region_names.LIMIT); -- 100

  FOR i IN r.country_names.FIRST .. r.country_names.LAST
  LOOP
    IF r.country_names.EXISTS(i) THEN -- test if exists
      dbms_output.put_line(r.country_names(i));
    END IF;
  END LOOP;
END;
/
```


Well I caught it
Now what?

Collection Methods – **PRIOR** and **NEXT**

PRIOR and **NEXT** are **functions** that let you **move backward** and **forward** in the collection. They **ignore** deleted elements, even if **DELETE** kept **placeholders** for them and are useful for **traversing sparse** collections. Given an **index** (**must exist**):

- **PRIOR**: If it **exists**, then **PRIOR** returns the index of the **preceding existing** element else **PRIOR** returns **NULL**
- **NEXT**: If it **exists**, then **NEXT** returns the index of the **succeeding existing** element else **NEXT** returns **NULL**

For an **associative array indexed** by **string** **PRIOR** and **NEXT** indexes are determined by key values (always sorted order).

If the collection c is a varray and the index exceeds c.LIMIT then c.PRIOR(index) returns c.LAST and c.NEXT(index) returns NULL.

c.PRIOR(c.FIRST) returns NULL for any collection c

c.NEXT(c.LAST) returns NULL for any collection c

```
DECLARE
    TYPE column IS TABLE OF INTEGER INDEX BY
PLS_INTEGER;
    TYPE row IS TABLE OF column INDEX BY PLS_INTEGER;
    mat row;
    tmat row;
    l STRING(100);
    i PLS_INTEGER;
    j PLS_INTEGER;
BEGIN
    mat := row( 1 => column(1=>1, 2=> 2, 3=> 3, 4=> 4),
                2 => column(1=>5, 2=> 6, 3=> 7, 4=> 8),
                3 => column(1=>9, 2=>10, 3=>11, 4=>12));

    dbms_output.put_line('Original Matrix');
    i := mat.LAST;
    WHILE i IS NOT NULL
    LOOP
        j := mat(i).LAST;
        WHILE j IS NOT NULL
        LOOP
            tmat(j)(i) := mat(i)(j);
            j := mat(i).PRIOR(j);
        END LOOP;
        i := mat.PRIOR(i);
    END LOOP;

    dbms_output.put_line('Transpose Matrix');
    i := tmat.FIRST;
    WHILE i IS NOT NULL
    LOOP
        l := '';
        j := tmat(i).FIRST;
        WHILE j IS NOT NULL
        LOOP
            l := l || tmat(i)(j) || ' ';
            j := tmat(i).NEXT(j);
        END LOOP;
        dbms_output.put_line(l);
        i := tmat.NEXT(i);
    END LOOP;

END;
/
```

```sql
CREATE OR REPLACE TYPE employees_vec AS VARRAY(20) OF varchar2(20);
/

CREATE TABLE EmployeesDeparments(
  numedept varchar2(30),
  angajati Employees_vec
);

DECLARE
  emps employees_vec;
  idd departments.department_id%TYPE := 110;
  dname departments.department_name%TYPE;
BEGIN
  SELECT department_name INTO dname FROM departments WHERE department_id =
idd;
  SELECT first_name||' '||last_name
  BULK COLLECT INTO emps
  FROM employees
  WHERE department_id = idd;

  INSERT INTO EmployeesDeparments values(dname, emps);
END;
/

SELECT * FROM EmployeesDeparments;

DROP TABLE EmployeesDeparments;
DROP TYPE employees_vec;
```

```
CREATE OR REPLACE TYPE jobs_nt AS TABLE OF varchar2(30);
/

CREATE TABLE EmployeesJobs(
  ename varchar2(40),
  ejobs jobs_nt
)
NESTED TABLE ejobs STORE AS ejobs_store;

set serveroutput on;
DECLARE
  ejobs jobs_nt;
  ide employees.employee_id%TYPE := 101;
  ename varchar2(40);
BEGIN
  SELECT first_name||' '||last_name INTO ename
  FROM employees WHERE employee_id = ide;

  WITH
    q1 as (SELECT e.employee_id, j.job_title
            FROM employees e
              INNER JOIN job_history jh ON jh.employee_id=e.employee_id
              INNER JOIN jobs j ON j.job_id = jh.job_id
            UNION
            SELECT e.employee_id, j.job_title
            FROM employees e
              INNER JOIN jobs j ON j.job_id = e.job_id)
  SELECT job_title BULK COLLECT INTO ejobs
  FROM q1
  WHERE q1.employee_id = ide;

  INSERT INTO EmployeesJobs values(ename, ejobs);

END;
/

SELECT * FROM EmployeesJobs;

DROP TABLE EmployeesJobs;

DROP TYPE jobs_nt;
```

Bibliography

Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference* 23ai, Oracle Corporation, 2024 [pdf]

Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 19c*, Oracle Corporation, 2022 [pdf]

Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 21c*, Oracle Corporation, 2022 [pdf]

Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 19c*, Oracle Corporation, 2020 [pdf]

Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 21c*, Oracle Corporation, 2021 [pdf]

Resources

- Flaticon:
  - https://www.flaticon.com/free-icon/time-record_1576640
  - https://www.flaticon.com/free-icon/dictionary_5402323
  - https://www.flaticon.com/free-icon/array_8093284
  - IStock Photo: https://www.istockphoto.com/ro/vector/verifica%C8%9Bi-marcajele-butonul-ui-cu-dos-%C8%99i-donts-plat-simplu-stil-tendin%C8%9B%C4%83-modern%C4%83-gm1163207144-319328344
- Memes:
  - https://i.pinimg.com/736x/f9/c6/24/f9c624561595995676ffec4d360257e7.jpg
  - https://ro.pinterest.com/pin/225186854727306/
  - https://www.dailydot.com/unclick/tim-eric-free-real-estate-meme-reddit/
  - https://www.reddit.com/r/funny/comments/hd51wn/giraffes_the_genesis/
  - https://pt.memedroid.com/memes/detail/3965907/i-am-at-my-limit-too
  - https://upload.wikimedia.org/wikipedia/commons/a/a5/Glazed-Donut.jpg
  - https://pt.memedroid.com/memes/detail/2104382